

## Answers for Week 4

---

### 1. Broadcasting

1. **Autolab** Write a Python function `standardize_rows` which receives three inputs: `data` with shape  $n \times d$ , `mean` with shape  $d$ , `std` with shape  $d$ . It must then subtract `mean` from each row of `data` and then divide by `std`. Finally, it must return the result. Do not use any NumPy functions.  
*Input:* Three arrays: `data` with shape  $n \times d$ , `mean` with shape  $d$ , `std` with shape  $d$ .  
*Output:* a new array `output` where `output[i] = (data[i] - mean) / std`.  
*Example:* For inputs

$$\text{data} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \text{mean} = [0.5 \quad 1 \quad 3] \text{ and } \text{std} = [1 \quad 2 \quad 3]$$

the output should be

$$\begin{bmatrix} 0.5 & 0.5 & 0 \\ 3.5 & 2 & 1 \end{bmatrix}.$$

---

**Answer:** Autolab exercise - answer omitted

---

2. **Autolab** Write a Python function `outer` that receives two vectors as input and returns the [outer product](#) of the two. Do not use any NumPy functions.  
*Input:* Two vectors as NumPy arrays of length  $n$  and  $m$  respectively.  
*Output:* The  $n \times m$  matrix giving the outer product.  
*Example:* The outer product of  $x = [1, 2]$  and  $y = [3, 4, 5]$  is

$$\begin{bmatrix} 3 & 4 & 5 \\ 6 & 8 & 10 \end{bmatrix}.$$

---

**Answer:** Autolab exercise - answer omitted

---

3. **Autolab** Write a Python function `distmat_1d` that receives two vectors as input and returns the distance matrix between them. Do not use for loops or any NumPy functions except for `abs`.

*Input:* Two vectors  $x$  and  $y$  of length  $n$  and  $m$  respectively.

*Output:* An  $n \times m$  matrix  $D$  where  $D_{ij} = |x_i - y_j|$ .

*Example:* For inputs  $x = [1, 2]$  and  $y = [3, 0.5, 1]$  the output should be

$$\begin{bmatrix} 2 & 0.5 & 0 \\ 1 & 1.5 & 1 \end{bmatrix}.$$

---

**Answer:** Autolab exercise - answer omitted

---

## 2. High Performance Haversine

In this exercise, we will expand on the example from section 2.2 in Fast Python about profiling code for the Haversine distance. Consider the following Python program:

```

1 import sys
2 import numpy as np
3
4 def distance_matrix(p1, p2):
5     p1, p2 = np.radians(p1), np.radians(p2)
6
7     D = np.empty((len(p1), len(p2)))
8     for i in range(len(p1)):
9         for j in range(len(p2)):
10             dsin2 = np.sin(0.5 * (p1[i] - p2[j])) ** 2
11             cosprod = np.cos(p1[i, 0]) * np.cos(p2[j, 0])
12             a = dsin2[0] + cosprod * dsin2[1]
13             D[i, j] = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
14
15     D *= 6371 # Earth radius in km
16     return D
17
18
19 def load_points(fname):
20     data = np.loadtxt(fname, delimiter=',', skiprows=1, usecols=(1, 2))
21     return data
22
23
24 def distance_stats(D):
25     # Extract upper triangular part to avoid duplicate entries
26     assert D.shape[0] == D.shape[1], 'D must be square'
27     idx = np.triu_indices(D.shape[0], k=1)
28     distances = D[idx]
29     return {
30         'mean': float(distances.mean()),

```

```

31         'std': float(distances.std()),
32         'max': float(distances.max()),
33         'min': float(distances.min()),
34     }
35
36
37     fname = sys.argv[1]
38     points = load_points(fname)
39     D = distance_matrix(points, points)
40     stats = distance_stats(D)
41     print(stats)

```

This program loads a set of latitude and longitude coordinates, computes a distance matrix using the Haversine distance, and finally prints some summary statistics.

The goal of this exercise is to optimize the program through the use of profiling and NumPy. For testing we use the same weather station data as in Fast Python. We have prepared smaller subsets for fast testing. You can find all the data and subsets at `/dtu/projects/02613_2025/data/locations/`.

1. **Autolab** Make a job script that runs the above program on the hpc queue. Request a single core and specify a CPU model so the results are repeatable. For the Autolab submission, assume that the input is always a file with path `input.csv`.

---

**Answer:** Autolab exercise - answer omitted

---

2. **Autolab** Modify your job script so it runs the program under the builtin `cProfile` profiler. What do you see? In what functions does the program spend the most time? \ Hint: See section 2.1.2 and 2.2.1 in Fast Python. \ Hint: Search for the name of your Python script in the profiler output to focus on the relevant functions.

---

**Answer:** Autolab exercise - answer omitted

---

3. **Autolab** Let us optimize the `distance_matrix` function. Using NumPy array operations and broadcasting, rewrite the function so there is only one loop instead of two nested ones.

---

**Answer:** Autolab exercise - answer omitted

---

4. Rerun the profiler. What has changed? Did it get faster? How much / little?

**Answer:** The exact times will depend on the chosen CPU and data subset. For Intel(R) Xeon(R) Gold 6226 CPU @ 2.70GHz and `locations_500.csv`, the profile output (filtering out all but our script) after optimizing `distance_matrix` is:

```

1 1      0.000      0.000      0.196      0.196 points.py:1(<module>)
2 1      0.026      0.026      0.026      0.026 points.py:21(distance_matrix
  )
3 1      0.000      0.000      0.008      0.008 points.py:66(load_points)
4 1      0.001      0.001      0.003      0.003 points.py:71(distance_stats)

```

The `distance_matrix` function is now significantly faster! I measured about 80x speedup, but the number will vary. However, it is still the slowest function of the bunch.

5. Let us now zoom in on the `distance_matrix` function using line profiling. Using the `line_profiler`, modify your Python program and job script to run with line profiling enabled on `distance_matrix`. Inspect the results. What do you see? On what lines should we focus? Hint: See section 2.2.2 in Fast Python.

**Answer:** Again, the exact times will vary depending on the chosen CPU, but the same lines should be the most time consuming on all CPUs. For Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz I get the following output:

```

1 Total time: 0.0329796 s
2 File: points.py
3 Function: distance_matrix at line 20
4
5 Line #      Hits          Time  Per Hit    % Time  Line Contents
6 =====
7      20                                  @profile
8      21                                  def
9      22      distance_matrix(p1, p2):
10     23          1          15.0      15.0      0.0      p1 = np.
11     24      radians(p1)
12     25          1           4.5       4.5      0.0      p2 = np.
13     26      radians(p2)
14     27          1           9.6       9.6      0.0      D = np.empty
15     28      (...)
16     29          500        168.3       0.3      0.5      for i in
17     30      range(len(p1)):
18     31          499       17275.1      34.6     52.4      dsin2 =
19     32      ...

```

```

15      28      499      6913.0      13.9      21.0      cosprod =
16      ...
16      29      499      1790.0      3.6      5.4      a = ...
17      30      499      4762.1      9.5      14.4      row = np.
      arctan2(...)
18      31      499      1841.4      3.7      5.6      D[i, :] =
      row
19      32
20      33      1      199.7      199.7      0.6      D *= 6371 #
      Earth ...
21      34      1      0.8      0.8      0.0      return D

```

From this, we see that we should focus on lines 27, 28 and 30 as they use the largest percentage of time.

6. Optimize `distance_matrix` to improve its performance. There are at least two optimizations you can make:

- Moving a repeated calculation out of the loop to pre-compute once.
- The fact that  $\arctan2(\sqrt{a}, \sqrt{1-a}) = \arcsin(\sqrt{a})$ . Re-run the profiler. Did it make the code faster? Did it change where time is spent in `distance_matrix`?

**Answer:** Instead of computing the cosine of individual elements every time in line 28, we can pre-compute it for all elements before the loop. This has two advantages:

- We do not re-compute the cosine of the `p2` elements every iteration, which is wasted work.
- We take advantage of NumPy array operations by only calling `np.cos` once instead of incurring the overhead of function calls every loop iteration.

Next, instead of computing the final result with `np.arctan2(np.sqrt(a), np.sqrt(1 - a))`, we can use the identity above and use `np.arcsin(np.sqrt(a))`. This means we only compute one square root, which saves time.

For me, with an Intel(R) Xeon(R) Gold 6226 CPU @ 2.70GHz and with all locations as input, these two optimizations reduced the runtime of `distance_matrix` from 12.6 seconds to 8.5 which is about 25% faster. Not mind blowing, but certainly worth the work.

7. **Autolab** Finally, let us see what happens if we eliminate for loops completely. Rewrite `distance_matrix` to perform all calculations with NumPy array operations and broadcasting. What happened? Did it get faster?

---

**Answer:** Autolab exercise - Code omitted.

Despite relying fully on efficient NumPy array ops, this code does not run faster than the optimized single loop version using an Intel(R) Xeon(R) Gold 6226 CPU @ 2.70GHz CPU and running on all locations. A likely reason is that the intermediate arrays it creates are too big, so we run into issues with memory speeds. We will explore this more in the next question.

---

8. Measure the performance in MFLOP/S for the optimized single loop and the no-loop version of `distance_matrix`. Use random points as input and vary the number of points from  $10^1$  to  $10^4$ . Plot the results and explain what you observe.
- 

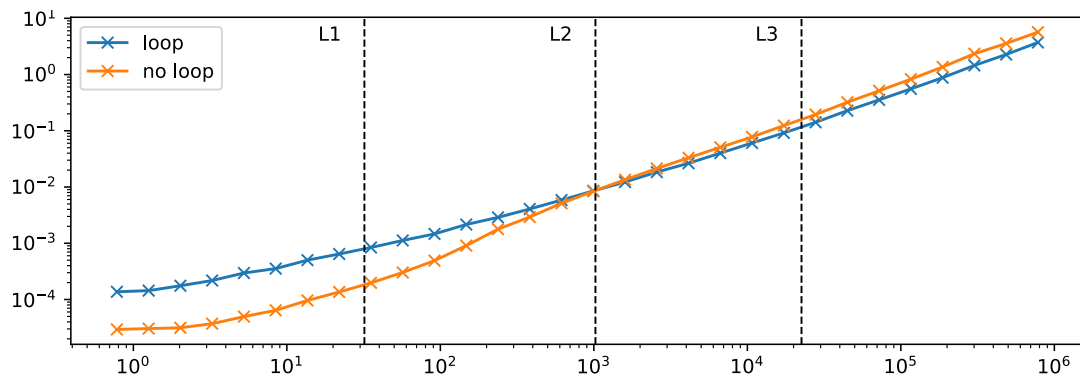
**Answer:** A script for performing the measurements may look like:

```
1 from time import perf_counter as time
2 import numpy as np
3
4 def distance_matrix_loop(p1, p2):
5     # Your code from exercise 2.3
6
7 def distance_matrix_noloop(p1, p2):
8     # Your code from exercise 2.7
9
10 loop_times = []
11 noloop_times = []
12 ns = np.logspace(1, 4, 30)
13 n_repeat = 5
14 for n in ns:
15     p1 = np.random.rand(int(n), 2)
16     p2 = np.random.rand(int(n), 2)
17     t = time()
18     for _ in range(n_repeat):
19         distance_matrix_loop(p1, p2)
20     loop_times.append((time() - t) / n_repeat)
21     t = time()
22     for _ in range(n_repeat):
23         distance_matrix_noloop(p1, p2)
24     noloop_times.append((time() - t) / n_repeat)
25
26 print('ns =', list(ns))
27 print('loop_times =', loop_times)
28 print('noloop_times =', noloop_times)
```

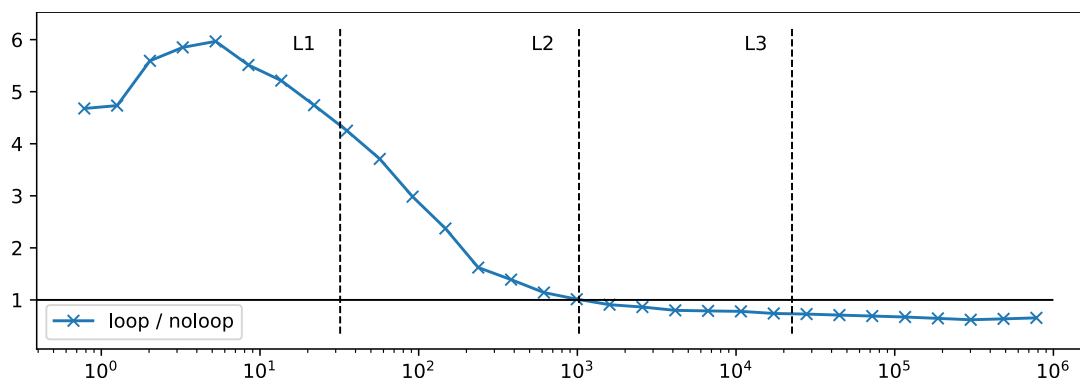
---

- a) Plot the run time in a loglog plot as a function of the size of the distance matrix in kilobytes. Include CPU cache sizes in your plot. Is one always faster? If yes, which one? If not, when do they switch?

**Answer:** Again, the exact numbers will vary with the CPU. For Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz, we get the following plots for the performance:



To make the difference clearer, we plot the ratio of the performance:



For small arrays, the version without loops is significantly faster but for large arrays the optimized single loop version is faster. The crossover happens when the distance matrix does not fit in the L2 cache. Note, this is not a rigorous analysis - for that, we would need to fully measure the memory use and cache misses. However, it does suggest that caches play a part. Finally, the **big lesson**: you cannot always predict what code will be fastest - you have to measure!